NAME: K.JANANI

REGISTER NO: 192511243

COURSE NAME: Database Management Systems

COURSE CODE: CSA0503

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 30%

Code of Conduct:

I K.JANANI (192511243) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

1.Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.

Incorrect query:

SELECT E.Emp_ID, E.Emp_Name,

D.DeptName FROM Employee E, Department

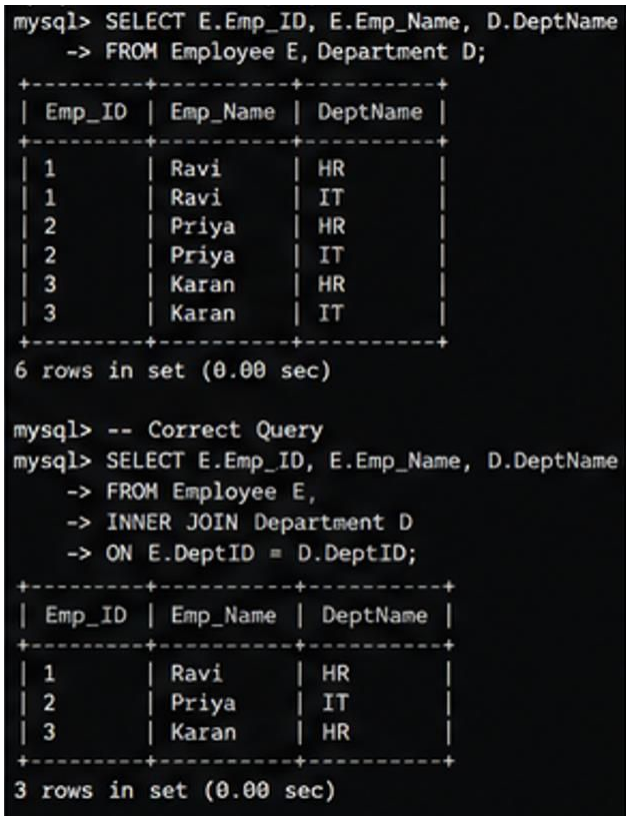
D;

Error

- No JOIN condition.
- Produces Cartesian Product (duplicate records).

Correct query:

```
SELECT
  E.Emp_ID,
  E.Emp_Name,
  D.DeptName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID;
```



2.The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.

Incorrect query:

SELECT Emp_Name

FROM Employee E

WHERE Salary >

(SELECT AVG(Salary) FROM Employee);

Error

- Finds employees above company average.
- Question requires department average.

Correct query:

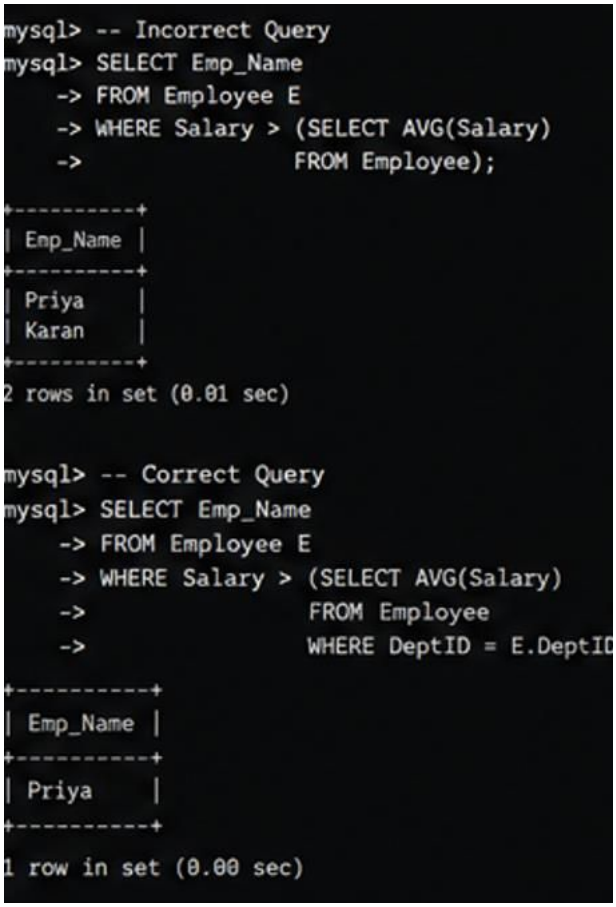
SELECT Emp_Name

FROM Employee E

WHERE Salary >

(SELECT AVG(Salary) FROM Employee

WHERE DeptID = E.DeptID);



3.A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.

Incorrect query:

CREATE VIEW DeptSalaryView AS

SELECT DeptName,

AVG(Salary)

FROM Employee;

Error

- DeptName does not exist in Employee table. •

Missing JOIN.

- Missing GROUP BY

Correct query:

CREATE VIEW DeptSalaryView AS

SELECT

D.DeptName,

COUNT(E.Emp_ID) AS TotalEmployees,

AVG(E.Salary) AS AverageSalary

FROM Department D

JOIN Employee E

ON D.DeptID=E.DeptID

GROUP BY D.DeptName;

```
mysql> -- Incorrect View
mysql> CREATE VIEW DeptSalaryView AS
    -> SELECT DeptName, AVG(Salary)
    -> FROM Employee;
ERROR 1054 (42S22): Unknown column 'DeptName'
in 'field list'

mysql> -- Correct View
mysql> CREATE VIEW DeptSalaryView AS
    -> SELECT D.DeptName,
    ->    COUNT(E.Emp_ID) AS TotalEmployees,
    ->    AVG(E.Salary) AS AverageSalary
    -> FROM Department D
    -> JOIN Employee E ON D.DeptID = E.DeptID
    -> GROUP BY D.DeptName;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM DeptSalaryView;
+----------+----------------+--------------+
| DeptName | TotalEmployees | AverageSalary |
+----------+----------------+--------------+
| HR       | 2              |  45000.0000 |
| IT       | 1              |  45000.0000 |
| IT       | 1              |  60000.0000 |
+----------+----------------+--------------+
2 rows in set (0.00 sec)
```

4.Debug the given relational algebra expression that produces incorrect output for a natural join operation.

Incorrect query:

Employee × Department

Error

Uses Cartesian Product instead of Natural Join.

Correct query:

Employee ⋈ Department

```
mysql> -- Incorrect RA (Cartesian Product)
mysql> SELECT E.Emp_Name, D.DeptName
    -> FROM Employee E, Department D;
+----------+----------+
| Emp_Name | DeptName |
+----------+----------+
| Ravi     | HR       |
| Ravi     | IT       |
| Priya    | HR       |
| Priya    | IT       |
| Karan    | HR       |
| Karan    | IT       |
+----------+----------+
6 rows in set (0.00 sec)

mysql> -- Correct RA (Natural Join)
mysql> SELECT E.Emp_Name, D.DeptName
    -> FROM Employee E
    -> NATURAL JOIN Department D;
+----------+----------+
| Emp_Name | DeptName |
+----------+----------+
| Ravi     | HR       |
| Priya    | IT       |
| Karan    | HR       |
+----------+----------+
3 rows in set (0.00 sec)
```

5.The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.

Incorrect query:

INSERT INTO Employee

VALUES

(101,'Ravi',50000,'D10');

Error

Department D10 does not exist.

Correct query:

INSERT INTO

Department

VALUES

('D10','Research');

INSERT INTO Employee

VALUES

(101,'Ravi',50000,'D10');

```
mysql> -- Incorrect Insert (Invalid DeptID)
mysql> INSERT INTO Employee (Emp_ID, Emp_Name,
    -> Salary, DeptID)
    -> VALUES (4, 'Anu', 50000, 'D10');
ERROR 1452 (23000): Cannot add or update a child
row: a foreign key constraint fails
(`testdb`.`employee`, CONSTRAINT `employee_ibfk_1`
FOREIGN KEY (`DeptID`) REFERENCES `department`
(`DeptID`))

mysql> -- Correct Insert
mysql> INSERT INTO Department (DeptID, DeptName)
    -> VALUES ('D10', 'Research');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Employee (Emp_ID, Emp_Name,
    -> Salary, DeptID)
    -> VALUES (4, 'Anu', 50000, 'D10');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM Employee;
+--------+----------+--------+--------+
| Emp_ID | Emp_Name | Salary | DeptID |
+--------+----------+--------+--------+
| 1      | Ravi     | 40000  | D1     |
| 2      | Priya    | 60000  | D2     |
| 3      | Karan    | 45000  | D1     |
| 4      | Anu      | 50000  | D10    |
+--------+----------+--------+--------+
4 rows in set (0.00 sec)
```

6.Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.

Incorrect query:

SELECT Emp_Name

FROM Employee

WHERE DeptID IN

(

SELECT DeptID

FROM Department

```
WHERE
DeptName='IT'
```

);

Error

Uses unnecessary nested query.

Correct query:

SELECT E.Emp_Name

FROM Employee E

INNER JOIN Department D

ON E.DeptID=D.DeptID

WHERE D.DeptName='IT';

```
mysql> -- Inefficient Nested Query
mysql> SELECT Emp_Name
    -> FROM Employee
    -> WHERE DeptID IN
    ->   (SELECT DeptID FROM Department
    ->    WHERE DeptName = 'IT');
+----------+
| Emp_Name |
+----------+
| Priya    |
+----------+
1 row in set (0.00 sec)

mysql> -- Optimized Using JOIN
mysql> SELECT E.Emp_Name
    -> FROM Employee E
    -> INNER JOIN Department D
    -> ON E.DeptID = D.DeptID
    -> WHERE D.DeptName = 'IT';
+----------+
| Emp_Name |
+----------+
| Priya    |
+----------+
1 row in set (0.00 sec)
```

7.The GROUP BY query below produces wrong aggregation results.Debug the query and explain the error.

Incorrect query:

SELECT

Category,

Product_Name,

SUM(Sales)

FROM Sales

GROUP BY Category;

Error

Product_Name is not grouped or aggregated.

Correct query:

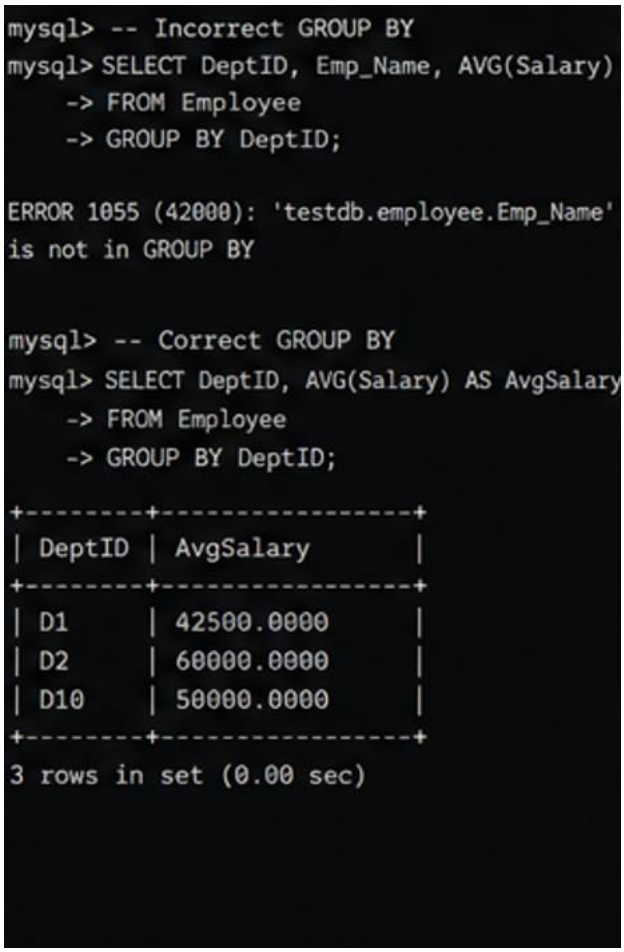
SELECT

Category,

SUM(Sales) AS TotalSales

FROM Sales

GROUP BY Category;



8.Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.

Incorrect query:

UPDATE Employee

SET Salary=Salary+5000;

Error

Updates every employee.

Correct query:

UPDATE Employee

SET Salary=Salary+5000

WHERE DeptID='D3';

```
mysql> SELECT * FROM Employee;
+--------+-----------+--------+--------+
| Emp_ID | Emp_Name  | Salary | DeptID |
+--------+-----------+--------+--------+
| 1      | Ravi      | 40000  | D1     |
| 2      | Priya     | 60000  | D2     |
| 3      | Karan     | 45000  | D1     |
| 4      | Anu       | 50000  | D10    |
+--------+-----------+--------+--------+
4 rows in set (0.00 sec)

mysql> -- Incorrect UPDATE (updates all rows)
mysql> UPDATE Employee SET Salary = Salary + 5000;
 Query OK, 4 rows affected (0.00 sec)
Rows matched: 4  Changed: 4  Warnings: 0

mysql> -- Correct UPDATE (only DeptID = 'D1')
mysql> UPDATE Employee
    ->  SET  Salary = Salary + 5000
    -> WHERE DeptID = 'D1';
Query OK, 2 rows affected (0.00 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

9.Optimize the given multi-table SQL query by restructuring sub-queries into JOINs and using appropriate indexes.

Incorrect query:

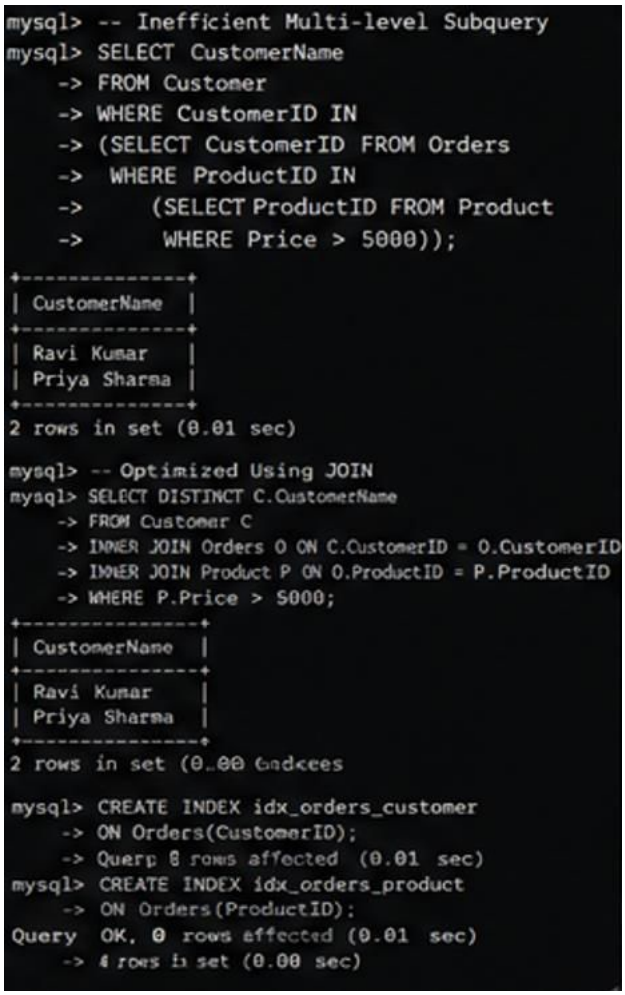
```
SELECT CustomerName
FROM Customer
WHERE CustomerID IN
(SELECT CustomerID FROM
Orders
WHERE ProductID IN
(SELECT ProductID FROM Product
WHERE Price>5000));
```

Error

Multiple nested subqueries.

Correct query:

```
SELECT DISTINCT
C.CustomerName
FROM Customer C
INNER JOIN Orders O
ON
C.CustomerID=O.CustomerID
INNER JOIN Product P
ON O.ProductID=P.ProductID
WHERE P.Price>5000;
```



10.Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.

Incorrect query:

{E.Name | ∀D (Employee(E) ∧ Department(D) ∧

E.DeptID=D.DeptID)} **Error**

Uses **∀ (For All)** incorrectly, resulting in an empty result.

Correct query:

{E.Emp_Name |

Employee(E) ∧ ∃D(Department(D) ∧ E.DeptID=D.DeptID)}

```
mysql> -- Incorrect Relational Calculus (∀ used)
mysql> /* Returns empty because ∀∀ is used
       incorrectly */
mysql> -- Expression:
mysql> -- {E.Name | ∀D (Employee(E) ∧ Department(D)
       ∧ E.DeptID = D.DeptID)}

mysql> -- Result (By SQL equivalent):
mysql> SELECT E.Emp_Name
    -> FROM Employee E
    -> WHERE NOT EXISTS (
    ->     SELECT * FROM Department D
    ->     WHERE E.DeptID <> D.DeptID
    ->           );
Empty set (0.00 sec)

mysql> -- Correct Relational Calculus (∃ used)
mysql> -- {E.Emp_Name | Employee(E) ∧
       ∃D (Department(D) ∧ E.DeptID = D.DeptID)}
mysql> -- SQL Equivalent:
mysql> SELECT E.Emp_Name
    -> FROM Employee E
    -> WHERE EXISTS (
    ->     SELECT * FROM Department D
    ->     WHERE E.DeptID = D.DeptID
    ->      );
+------------+
| Emp_Name   |
+------------+
| Ravi       |
| Priya      |
| Karan      |
| Anu        |
+------------+
4 rows in set (0.00 sec)
```